Secure File Transfer Monitoring System

Project report and verification record

Blue Team · Data Loss Prevention · File Integrity Monitoring

Vansh Dhawan

First internship project · Unified Mentors

[Supervisor]

[Date]

Contents

Contents 2
Abstract 3
1. Introduction and problem statement 4
2. Objectives and learning outcomes 5
3. Scope and tools 6
4. System architecture 7
5. Workflow and policy branches 8
6. Implementation modules 9
7. Detection rules 10
8. Testing and verification 11
9. Demonstration results 12
10. Integrity evidence 13
11. Limitations 14
12. Future work 15
13. Conclusion 16
References 17
Appendix A. Policy sample 18
Appendix B. Command reference 19
Appendix C. Traceability and evidence 20

Abstract

This report describes a local Python monitor that records file-system changes, classifies configured paths, compares SHA-256 hashes, applies rules R0–R6, and creates audit reports. The deterministic demo exercises ordinary and approved copies, simulated external destinations, file changes, deletion, and a 200-file burst. The final captured demo contains 210 event rows. The tool is an audit aid: it does not block transfers or observe network protocols.

The final automated suite reports 20 passed in 19.23s. Two clean runs, independent hashes, sabotage checks, and the source limitations are detailed below.

1. Introduction and problem statement

A file can leave an approved folder, change after transfer, or disappear without a single readable audit trail. Security teams need to see which file changed, where it went, when it happened, and which policy rule applies. This project demonstrates that record using local folder notifications and a small JSON policy.

The monitor uses path labels to represent USB storage, cloud sync folders, and network shares. These are configured directories, not device-level or network-level detectors. The system records evidence for review; it does not prevent an operating-system action.

2. Objectives and learning outcomes

Objective	Demonstration
Activity logging	Timestamped JSONL records include source, destination, user, process, hashes, and policy result.
DLP-style checks	R1/R2 mark sensitive transfers outside approved locations and into configured zones.
Integrity	SHA-256 compares baseline and observed content; R3 flags a mismatch.
Alerting and audit	Rules write console and log findings; reports summarize the timeline.
Testing	Unit and watcher tests plus deterministic scenarios check behavior.

A file-system transfer is a create, copy-like, move, delete, or modify event in a watched folder. A data-loss prevention rule applies policy to a file and its destination. A cryptographic hash is a content fingerprint: the same bytes should produce the same SHA-256 value. Monitoring records activity; auditing reviews those records to decide whether an action needs investigation.

3. Scope and tools

The implementation uses Python, watchdog, pathlib, hashlib, argparse, JSON Lines, pytest, python-docx, python-pptx, Pillow, and matplotlib. The runtime writes local files and uses no database, web application, or external service. Demo data is synthetic and contains no real credentials or customer records.

The project was exercised in Windows PowerShell from a OneDrive-synchronized folder whose path contains spaces and parentheses. Linux and macOS were not available for direct observation. Windows open/read auditing is outside the implementation.

In scope	Out of scope
Configured folder monitoring, classification, hashes, alerts, reports.	Blocking or quarantining a transfer.
Folder paths as USB, cloud, and share examples.	Hardware-level USB detection or cloud APIs.
Best-effort process field.	Reliable attribution when OS data is unavailable.

4. System architecture

The observer forwards filesystem notifications to a worker pool. The monitor classifies the event, checks baseline state, hashes stable files, and appends a structured record. Rule findings also go to the alert log. The report builder reads the event log and writes Markdown, HTML, and CSV views.

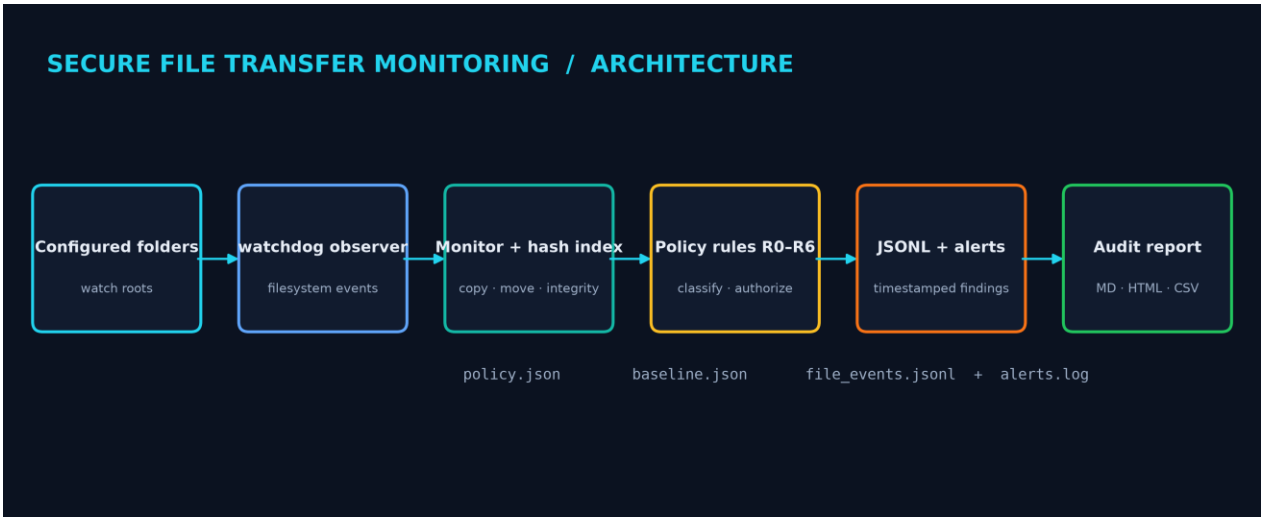


Figure 1. Monitoring modules and evidence flow.

The baseline maps each configured file to SHA-256, optional MD5, size, and modification time. The path index supports integrity checks; the hash index helps infer copies. A copy is inferred from notifications and matching content, not from a native copy signal supplied by watchdog.

5. Workflow and policy branches

The monitor classifies every event. If no sensitive or external policy condition applies, it records ordinary activity. A trigger leads to hash and authorization checks. A disallowed user produces R6; a content mismatch produces R3. The event is logged on each branch so the report can preserve a complete timeline.

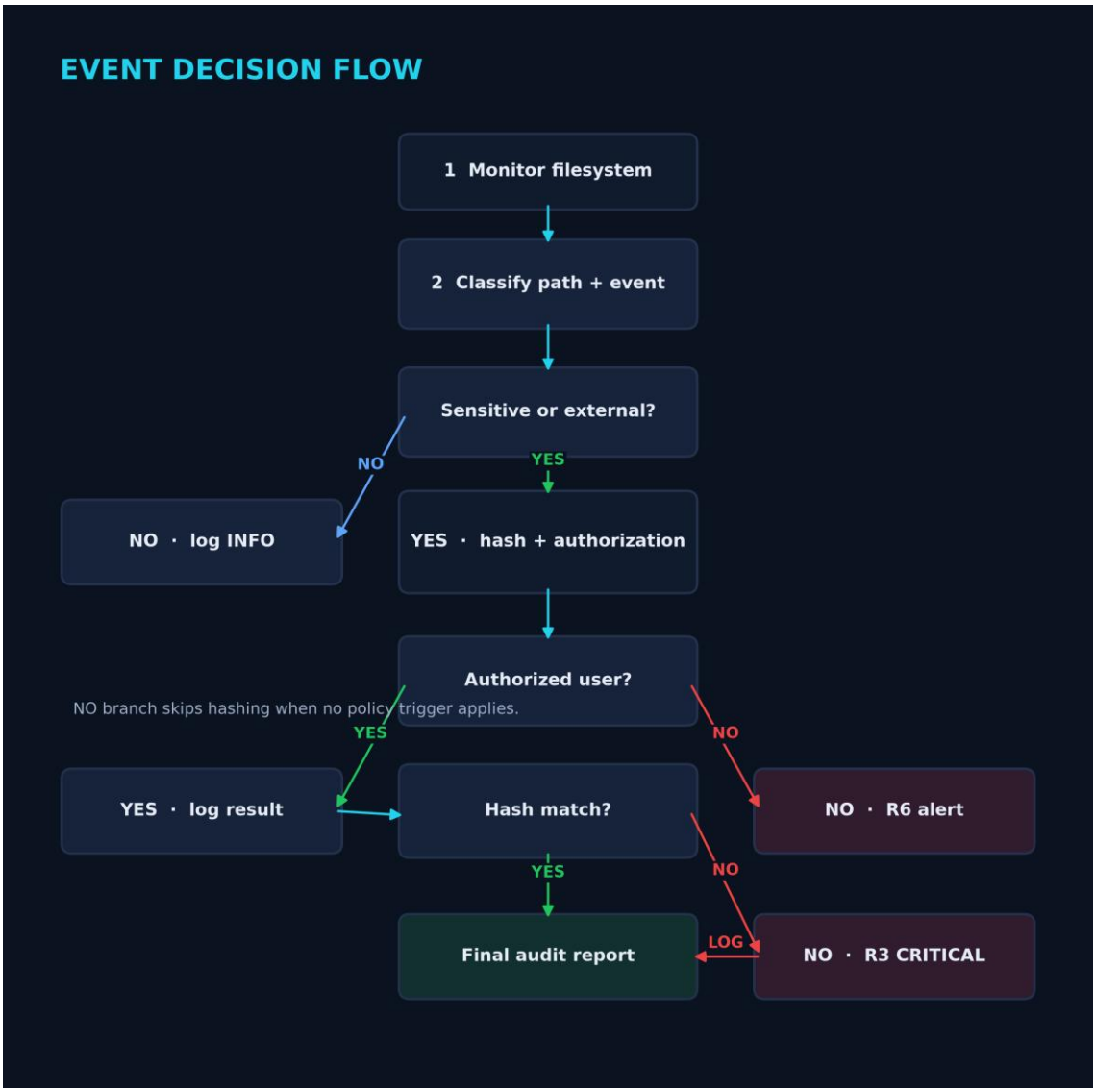


Figure 2. Classification, authorization, and integrity decisions.

6. Implementation modules

main.py Argparse dispatch for hash, baseline, monitor, report, verify, and demo.

hasher.py Chunked hashes and stable-file polling.

baseline.py Initial hash and metadata index.

policy.py Path resolution, sensitive names, zones, authorization, and rules.

handler.py Watchdog adapter for filesystem notifications.

monitor.py Indexes, copy inference, rename correlation, and burst aggregation.

logger.py Thread-safe JSONL and alert output with optional ANSI color.

report.py Markdown, HTML, and CSV summaries and timelines.

demo/run_demo.py Nine synthetic scenarios with event checks.

7. Detection rules

Rule	Condition	Severity / result
R0	Routine or approved activity	INFO
R1	Sensitive copy or move outside approval	HIGH
R2	Transfer into configured USB, cloud, or share zone	MEDIUM or HIGH
R3	Observed SHA-256 differs from indexed content	CRITICAL
R4	Configured count exceeded in one time window	HIGH; one summary per window
R5	Sensitive file deletion	HIGH
R6	Sensitive event by user outside allow-list	HIGH

Rules are additive: a sensitive USB transfer can match R1 and R2. The event row stores combined rule IDs and the strongest severity. The alert log stores a separate line for each finding.

8. Testing and verification

Check	Run 1	Run 2
Automated tests	20 passed in 19.23s	20 passed in 13.62s
Acceptance	16/16 checks	16/16 checks
Event rows	210	210
Event types	BULK=1, COPY=5, CREATE=200, DELETE=1, MODIFY=2, MOVE=1	BULK=1, COPY=5, CREATE=200, DELETE=1, MODIFY=2, MOVE=1
Severity totals	CRITICAL=2, HIGH=6, INFO=202	CRITICAL=2, HIGH=6, INFO=202

Tests cover hash vectors, classification and authorization, integrity match/mismatch, copy inference, one alert per bulk window, report generation, watcher delivery, stable writes, event ordering, process path matching, and privacy-safe external paths. The sabotage record documents three deliberate faults and the failures they caused.

9. Demonstration results

The nine scenarios cover normal copy, approved backup, USB copy, cloud copy, move, altered share copy, in-place tampering, sensitive deletion, and a 200-file burst. Every item is synthetic. Final logs and acceptance files are the source for the counts below.

Measure	Run 1	Run 2
Event records	210	210
Event types	BULK=1, COPY=5, CREATE=200, DELETE=1, MODIFY=2, MOVE=1	BULK=1, COPY=5, CREATE=200, DELETE=1, MODIFY=2, MOVE=1
Severity	CRITICAL=2, HIGH=6, INFO=202	CRITICAL=2, HIGH=6, INFO=202
Acceptance	16/16 checks	16/16 checks



Figure 3. USB alert from final log.

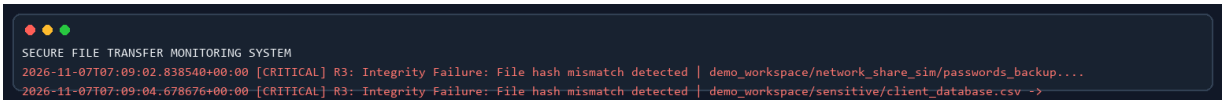


Figure 4. Integrity mismatch from final log.



Figure 5. Aggregated 200-file alert.

10. Integrity evidence

File	SHA-256
altered_copy.txt	71f06f874c7584504ea350174c5eae3c193136f9f947cf5e00cd19c4f3cc0157
correct_copy.txt	6ce3452e9266c630f079baaa8a4f0f18576c11bc1857a94b4da5ee7a70af0118
original.txt	6ce3452e9266c630f079baaa8a4f0f18576c11bc1857a94b4da5ee7a70af0118
verify_match.txt	fb902f661b5e7f38be9401f9997911a6e3cc65272441f49faa64c8ffced41bf2
verify_mismatch.txt	ee3454e12c1ae69528540b373bdffa15247f68b6cd570cc3d6e95145c6fa58c9

PowerShell Get-FileHash independently checked five files. The original and correct copy share a digest. The altered copy differs. The project verify command prints MATCH for the correct pair and MISMATCH for the altered pair.

```
python main.py verify evidence/integrity/original.txt evidence/integrity/correct_copy.txt
python main.py verify evidence/integrity/original.txt evidence/integrity/altered_copy.txt
```

11. Limitations

Limit	Effect
No enforcement	The monitor reports activity but cannot stop or reverse it.
Folder-based zones	Zone labels depend on configured paths; there is no hardware or network inspection.
No open/read audit	Windows notifications do not show every file access; R6 applies to observed events.
Process field	All final demo records say unknown; psutil is not installed here.
Platform variability	Windows was checked; other systems need their own verification.
OneDrive sync	Sidecar and rename notifications can add noise; desktop.ini is ignored by default.

12. Future work

A production extension could add Windows-specific access auditing, removable-drive identity, cloud-provider events, and SIEM forwarding. Deployment should also define log retention, access controls, alert ownership, and an incident review process. These features are outside this project and have not been tested.

13. Conclusion

The project turns configured file-system changes into structured evidence. Hash comparisons identify changed content, policy rules mark risky paths and users, and the report ties individual events to a summary. The repeatable demo and tests show behavior within the stated path-based scope.

For submission, replace the four identity placeholders, review the limits, and run the demo on the presentation computer before recording.

References

- Python pathlib and hashlib: https://docs.python.org/3/
- watchdog: https://python-watchdog.readthedocs.io/
- pytest: https://docs.pytest.org/
- python-docx: https://python-docx.readthedocs.io/
- python-pptx: https://python-pptx.readthedocs.io/

Appendix A. Policy sample

```
{
  "watched_dirs": ["demo_workspace"],
  "sensitive_dirs": ["demo_workspace/sensitive"],
  "restricted_extensions": [".key", ".pem", ".kdbx"],
  "restricted_name_keywords": ["password", "salary", "confidential"],
  "allowed_destinations": ["demo_workspace/approved_backup"],
  "external_zones": {
    "USB": ["demo_workspace/usb_drive_sim"],
    "CLOUD": ["demo_workspace/cloud_sync_sim"],
    "NETWORK_SHARE": ["demo_workspace/network_share_sim"]
  },
  "authorized_users": [],
  "bulk_transfer": {"threshold_files": 50, "window_seconds": 60},
  "hash_algorithms": ["sha256", "md5"],
  "ignore_patterns": ["*.tmp", "*.swp", "~$*", ".DS_Store", "desktop.ini"]
}
```

Replace the sample folders with local paths. Keep approved destinations narrow and review the allow-list before monitoring sensitive files.

Appendix B. Command reference

```
usage: main.py [-h] {baseline,monitor,report,verify,hash,demo} ...

Secure File Transfer Monitoring System

positional arguments:
  {baseline,monitor,report,verify,hash,demo}

options:
  -h, --help            show this help message and exit
```

Command	Purpose
python main.py baseline	Create a hash baseline.
python main.py monitor	Watch configured folders until Ctrl+C.
python main.py report	Rebuild reports from the current log.
python main.py hash FILE	Print SHA-256 and MD5.
python main.py verify SOURCE COPY	Compare SHA-256; mismatch exits 1.
python main.py demo --clean --pace 3	Run the demo with recording pauses.
python -m pytest -q	Run the automated suite.

Appendix C. Traceability and evidence

See docs/TRACEABILITY_MATRIX.md for the requirement-to-test links. Each final run folder contains its console transcript, logs, reports, and acceptance result. The evidence/final directory also stores both test outputs, independent hashes, negative acceptance, sabotage checks, and the clean-clone record.